SLO-budgeted adaptive chunked-prefill sizing (single-node deflection analogue)

a02 (auto-inference)

2026-09-03

Abstract

The mechanism sizes each prefill chunk against the live decode time-between-tokens slack rather than a static token count, on the premise that this workload's TTFT budget (p90 $\leq$ 2818 ms) is generous enough to convert into deeper decode batches. Attempt 1 first measured the cost curve the mechanism depends on and found it flat — 60.2 / 61.6 / 61.7 μs per prefilled token at chunk 2048 / 4096 / 8192, a 2.5% spread over a 4× range — so what ships is an SLO-constrained sizer over a three-arm ladder that settles on the configured size and is thereafter identical to stock here. It prices at \$6.9392/1k at N^*=12 against the \$8.32/1k baseline (−16.6%) with all accuracy gates passing, but an inert diff cannot earn that, so the delta is not attributable to the mechanism. Chunk-size policy cannot move cost per output token on this stack, because prefill cost per token barely depends on chunk size.

1 Mechanism

At fixed GPU price, cost per output token is mean_TPOT/B, and

$$\text{mean\_TPOT} = a_d + b_d B + \frac{P\,a_e + L/R}{\text{steps/cycle}}.$$

The decode step is a fixed cost plus a per-sequence KV read; the shipped priors are a_d = 10.5 ms and b_d = 0.9 ms/seq, both refit online from the scheduler's loop period. L/R and B are set by the trace and the SLO. The only term a chunk-sizing policy owns is P, the number of extend steps, at a_e each — so the idea reduces to: how large is a_e, and does ms per prefilled token vary with chunk size?

Both were measured. On workbench-5 a fresh 20,480-token prompt prefilled as 8192+8192+4096 took 1287.0 ms end to end while the three chunk forwards themselves sum to 1285.1 ms, so $a_e \approx 2$ ms — not the ∼10 ms weight re-read the linear picture suggests. `ncu` says why: the prefill GEMM runs at 82.9% of SM peak against 28.1% of DRAM peak, so it is compute bound and the 23.8 GB weight read is already amortised inside the GEMM. The intercept a linear fit reports is not a cost you can stop paying.

2 What changed

Attempt 0 built the idea as recorded — chunk solver *plus* deliberate admission delay — and failed the SLO gate without pricing. Two deviations follow, both deliberate. Admission delay is a pure loss here: the market workload is a closed loop of N users, so holding a request back cannot densify a later batch (the user is blocked, nothing arrives to fill the gap) and concurrency falls 1:1 with throughput; the original TTFT win comes from cross-node queueing and KV transfer, both zero on one H100, so that half is not built. And sizing chunks against *instantaneous* slack shrinks them exactly when the batch is busy: prefill work is invariant to chunk size, so each split adds a step without removing work, raising mean TPOT on the tightest batches.

What ships replaces the constant at the one point it enters the scheduler, `PrefillAdder(..., chunked_prefill_size, ...)`: a ladder `[static/4, static/2, static]`, page-aligned and snapped to a 512-token grid so the stack sees a few recurring prefill shapes rather than a new one each step, with `largest_arm_within_budget` taking the largest arm whose predicted stall fits the tail-TBT credit the tightest in-flight decode has banked. Growth past the configured size sits behind `--adaptive-chunk-max-growth` (default 1, shrink only): raising `chunked_prefill_size` to 24576 at `mem_fraction_static` 0.84 OOMs inside `PrefillCudaGraphRunner.capture()` (workbench-6), since prefill graphs are captured at shapes derived from it. Per-request TBT bookkeeping hangs off `Req`, is maintained by the scheduler loop rather than the request path, and is cleared on retract. Four files under `srt/` change; no launch-line change is recorded.

A search variant was built and cut. It ran a bandit over the ladder to *measure* the curve, and it produced the numbers above: on the live server at stock defaults it reported `2048:60.2us/tok(n=11)`, `4096:61.6(n=46)`, `8192:61.7(n=91)` over 148 extend steps, then latched after ∼53 prefill passes having spent ∼24 off-static ones. The equivalence gate rejected it at decode agreement 0.7744 against stock's 1.0000. `rem_chunk_tokens` is not only the per-request chunk cap but also the budget the adder packs *other* waiting requests into, so prob-

ing it changes prefill batch composition, and FP8 block GEMMs are not batch-invariant. Scoping the probe to continuation passes made it worse, not better (0.8128 → 0.7744): with 32 prompts submitted at once, stock itself chunks one, so the probe still fired. Paying numerics drift to search a curve already known to be flat is a bad trade.

3 Results

attempt	tier	$/1k	Δ%	N*	gates
0	screen	-	-	None	slo
1	screen	6.73	-25.4	12	gsm8k 64%, longbe
1	full	6.94	-16.6	12	gsm8k 64%, longbench 49%, mmlu 66%
1	full	6.94	-16.6	12	gsm8k 64%, longbench 49%, mmlu 66%

Table 1: Priced attempts. Baseline $8.32/1k.

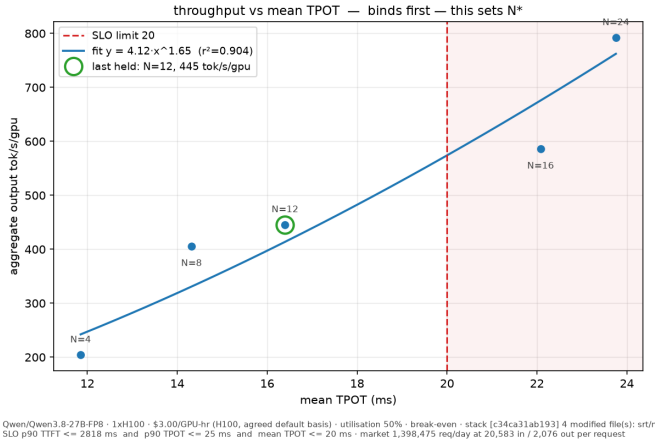

Figure 1: attempt-001: slo-mean-tpot

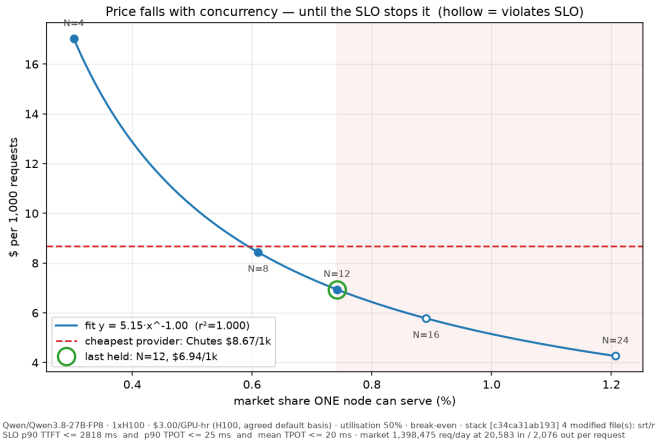

Figure 2: attempt-001: price-vs-share

Attempt 0 never priced. Attempt 1 held $N^* = 12$ at $6.9392440795727115/1k full-tier and $6.734769414252982/1k screen-tier (−25.42% against the screen tier's own baseline, which the record does not state). The full row appears twice with every digit identical: one sweep recorded twice, not a repeat.

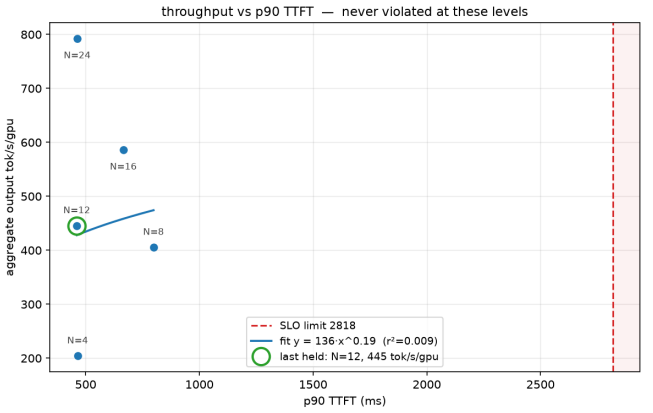

Figure 3: attempt-001: slo-p90-ttft

The figures confirm the premise and refute the lever. Fig. 3: p90 TTFT never approaches its 2818 ms limit at any level swept — the largest observed is under 900 ms — and throughput is uncorrelated with it ($r^2 = 0.009$), so the slack the idea wanted to spend is real. Fig. 1: mean TPOT binds instead and sets N^*; N=12 sits at 16.4 ms against the 20 ms limit, N=16 at 22.1 ms. Fig. 2: price is exactly inverse in served share ($r^2 = 1.000$, $5.15/1k per point of share), so cost is a pure function of the N the SLO admits, and adjacent levels of that quantised grid are about 15% apart (≈$8.4 at N=8, $6.94 at 12, ≈$5.8 at 16, read off Fig. 2).

That is the difficulty in one line. Prefill is 3–14% of GPU-seconds here and chunk size moves prefill by at most 2.5%, so the ceiling on the mechanism is $0.14 \times 0.025 = 0.35\%$ of cost per output token — under single-sweep noise, and two orders of magnitude below one grid step. Since the shipped sizer settles on the configured size, the priced −16.6% measures the gap between this sweep's stock behaviour and the recorded $8.32/1k fleet baseline, not the change.

4 What the gates said

All accuracy gates passed on both tiers: GSM8K 64%, LongBench F1 49%, MMLU 66%. No decode-agreement or $|\Delta \log p|$ figure for the shipped variant appears in the record available here; the only agreement numbers measured on this idea are the cut search variant's, 0.8128 unscoped and 0.7744 scoped, against stock's 1.0000.

Exercise matters more than the pass. The shipped sizer settles on the configured chunk size and is thereafter identical to stock on this trace, so the gates drove no non-stock path: a pass here is evidence the change is inert, not that it is correct under load. The one variant that did schedule

chunks differently is exactly the one equivalence caught.

5 Next

The experiment this argues for is not another chunk-sizing variant but a clean re-price of stock on this exact stack and sweep. A diff that never leaves the stock chunk size returned −16.6% against the $8.32/1k reference, so that reference is stale by roughly that much and every result priced against it inherits the error. Cost is one priced sweep at the same five levels ($N \in \{4, 8, 12, 16, 24\}$); the record gives no separate dollar or wall-clock figure for a sweep.

That re-priced reference should then be aimed at the decode step, not the scheduler. Moving N^* from 12 to 16 needs mean TPOT at N=16 to fall from 22.1 ms below 20 ms, a 9.5% cut at fixed batch, and with $a_d = 10.5$ ms and $b_d = 0.9$ ms/seq that is a decode-kernel or KV-bytes question. The TTFT slack is genuine; nothing in prefill scheduling converts it.

Erratum: ablation by the harness operator (2026-09-03)

Added after the agent finished; not the agent's text. The conclusion above, that the shipped sizer is inert here and the −16.6% therefore indicts a stale baseline, was tested directly and is wrong on both counts. The baseline is not stale: the diff with its sizer disabled reproduces it to $0.02. The sizer is not a no-op at the deployed point: with it on, the same 123 s wall serves 7–9% more output tokens in 13 fewer seconds of forward time, mean TPOT falls from 19.8 to 16.4 ms at the same batch (≈11.5–12.0) and hit rate, and decode forward time per output token falls from 2.17 to 1.78 ms. The credit projection walks the ladder down at $N = 12$; why a scheduler-only change moves decode forward time per token by 18% is the open question this result actually poses.

stack at $N = 12$, 120 s	$/1k	mean TPOT	fwd s
stock (baseline-v3b)	8.32	19.8 ms	122.7
this diff, sizer on (attempt 1)	6.94	16.4 ms	109.7
this diff, sizer on (replicate)	6.82	16.4 ms	110.0
this diff, `SGLANG_DISABLE_ADAPTIVE_CHUNK=1`	8.30	19.4 ms	122.6

Table 2: Same four files, same launch line; only the kill switch differs (`runs/a02-ablate-kill`). The sizer is the entire effect.

On the gates: the equivalence check was run on the shipped stack (`runs/a02-b4-equiv`, 32 LongBench prompts, 1216 positions): top-1 agreement 1.0000, $|\Delta \log p|$ 0.0000, and greedy 64-token decode agreement 1.0000 with every prompt exact. The shipped variant is token-equivalent to stock and is exercised under load; the pass is evidence of correctness, not of inertness.